

数値解析法 I 第2回講義

計算と誤差

計算と誤差

数値計算法：コンピュータ内で実現・使用

疑問：コンピュータで数値を正しく計算できるのか？



Javaで $98.7 + 78.9$ を
実行させるプログラムを作成

計算結果：177.600000000000002

コンピュータで求められる値：真値ではなく近似値

近似値と真値に違いについて知っておく必要有

誤差 (error)

誤差と有効桁数

定義1.1 (誤差, 絶対誤差, 相対誤差)

$\hat{x} \in \mathbb{R}$ を x の近似値とする. このとき

$\hat{x}$ の x に対する誤差: $e(\hat{x}) = \hat{x} - x$.

また $|e(\hat{x})|$ を絶対誤差, $e_r(\hat{x}) = e(\hat{x})/x$ ($x \neq 0$) を相対誤差という.

例) $x = 177.6$, $\hat{x} = 177.600000000000002$ とすると

$|e(\hat{x})| = 2.00 \times 10^{-14}$, $e_r(\hat{x}) \approx -1.13 \times 10^{-16}$ (−0.113 フェムト%)

注) x , $\hat{x}$ のスケールに $e(\hat{x})$: 依存, $e_r(\hat{x})$: 依存しない.

➡ 近似値の誤差評価: 相対誤差 $e_r(\hat{x})$ が用いられることが多い

先程の例： $x = 177.60000000000000$
 $\hat{x} = 177.600000000000002$ 16桁一致している

近似値が何桁一致しているか：理解し易く、かつ重要



定義1.2 (有効桁数)

$-\log_{10} |e_r(\hat{x})|$: 近似値 $\hat{x}$ の 10 進有効桁数 (または有効桁数).

例) $x = 177.6$, $\hat{x} = 177.600000000000002$ とすると

$$\text{有効桁数} = -\log_{10} |e_r(\hat{x})| \approx -\log_{10}(1.13 \times 10^{-16}) \approx 15.95$$

$$e(\hat{x}) = \hat{x} - x, \quad e_r(\hat{x}) = e(\hat{x})/x, \quad -\log_{10} |e_r(\hat{x})|$$

問題点：計算するには真値 x が必要

数値解析法：**誤差限界**を求めることが多い

定義1.3 (誤差限界)

$$|e(\hat{x})| \leq \varepsilon(\hat{x}), \quad |e_r(\hat{x})| \leq \varepsilon_r(\hat{x})$$

を満たす $\varepsilon(\hat{x}), \varepsilon_r(\hat{x})$ を**誤差限界**，**相対誤差限界**と呼ぶ。

注) 数値解析法の誤差評価 = 誤差限界を求める (大雑把にいうと)。

注) 有効桁数: $-\log_{10} \varepsilon_r(\hat{x})$ で代用されることが多い。

桁落ち，情報落ち

$$x = 1.00000002, \quad \hat{x} = 1.000000001 \quad (x \text{ の近似値})$$

$$\hookrightarrow e_r(\hat{x}) \approx -1.00 \times 10^{-7}, \quad \text{有効数字} \approx 7.00 \text{ 桁}$$

$$y = 1.00000001, \quad \hat{y} = 0.99999999 \quad (y \text{ の近似値})$$

$$\hookrightarrow e_r(\hat{y}) \approx -2.00 \times 10^{-7}, \quad \text{有効数字} \approx 6.70 \text{ 桁}$$

ところが...

$$x - y = 0.00000001, \quad \hat{x} - \hat{y} = 0.00000002 \quad (x - y \text{ の近似値})$$

$$\hookrightarrow e_r(\hat{x} - \hat{y}) = 1.00, \quad \text{有効数字} = 0 \text{ 桁}$$

桁落ち：近接する2つの数の減算で有効桁数が失われる現象

例) $ax^2 + bx + c = 0$ ($a \neq 0$) の解を近似計算

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

によりプログラム作成

$a = 1, b = 10^9, c = 1$ のとき

$$x_1 = 0.0000 \cdots 0 \times 10^0$$
$$(-1.0000 \cdots \times 10^{-9})$$

$$x_2 = -1.0000 \cdots 0 \times 10^9$$
$$(-9.9999 \cdots \times 10^8)$$

※青字は真値

x_1 で桁落ち発生

$a = 1, b = -10^9, c = 1$ のとき

$$x_1 = 1.0000 \cdots 0 \times 10^9$$
$$(-9.9999 \cdots \times 10^8)$$

$$x_2 = 0.0000 \cdots 0 \times 10^0$$
$$(1.0000 \cdots \times 10^{-9})$$

※青字は真値

x_2 で桁落ち発生

桁落ちの原因

$b^2 \gg 4ac$ により $\sqrt{b^2 - 4ac} \approx b$ となったため、 $b \geq 0$ のときは x_1 ， $b < 0$ のときは x_2 の分子において近接する 2 つの数の減算が発生



上手く有理化して...

True

$b \geq 0$

False

$$x_1 = \frac{-2c}{b + \sqrt{b^2 - 4ac}}$$
$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$
$$x_2 = \frac{-2c}{b - \sqrt{b^2 - 4ac}}$$

「有理化」を利用した方法によりプログラミング

$a = 1, b = 10^9, c = 1$ のとき

$$x_1 = -1.0000 \dots 0 \times 10^{-9}$$

$$(-1.0000 \dots \times 10^{-9})$$

$$x_2 = -1.0000 \dots 0 \times 10^9$$

$$(-9.9999 \dots \times 10^8)$$

※青字は真値

$a = 1, b = -10^9, c = 1$ のとき

$$x_1 = 1.0000 \dots 0 \times 10^9$$

$$(-9.9999 \dots \times 10^8)$$

$$x_2 = 1.0000 \dots 0 \times 10^{-9}$$

$$(1.0000 \dots \times 10^{-9})$$

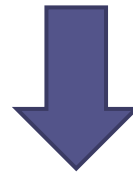
※青字は真値

桁落ちが発生しなくなった

理論的に正しい公式：数値計算での使用には注意が必要

◆ 情報落ち

例) $x = 1.234 \times 10^{10}$ と $y = 5.678 \times 10^{-7}$ をプログラムで加算



倍精度で計算させると

計算結果: $x + y = 1.234 \times 10^{10}$

y が加えられていないことと同じ結果



原因: 大きさが極端に違う数同士で加算したため

情報落ち

大きさが極端に違う数値同士で加減算を行ったとき、
小さいほうの数値の下位桁が失われる現象

例) 無限級数 $S = \sum_{k=1}^{\infty} \frac{1}{k^2} = \frac{\pi^2}{6} = 1.6449340668 \dots$ の近似計算

$$S_n = \left(\dots \left(\left(\frac{1}{1^2} + \frac{1}{2^2} \right) + \frac{1}{3^2} \right) + \dots + \frac{1}{n^2} \right)$$

によりプログラム作成



$$S_2 = 1.2500000 \quad (|S - S_2| \approx 3.9493406 \times 10^{-1})$$

.....

$$S_{2048} = 1.6444435 \quad (|S - S_{2048}| \approx 4.9054623 \times 10^{-4})$$

$$S_{4096} = 1.6447253 \quad (|S - S_{4096}| \approx 2.0873547 \times 10^{-4})$$

$$S_{8192} = 1.6447253 \quad (|S - S_{8192}| \approx 2.0873547 \times 10^{-4})$$

$$S_{16384} = 1.6447253 \quad (|S - S_{16384}| \approx 2.0873547 \times 10^{-4})$$

..... 近似値が増加しない (情報落ち発生)

情報落ちを防ぐため

$$S_n = \left(\cdots \left(\left(\frac{1}{n^2} + \frac{1}{(n-1)^2} \right) + \frac{1}{(n-2)^2} \right) + \cdots + \frac{1}{1^2} \right)$$

によりプログラム作成



情報落ちは無発生

$$\begin{aligned} S_{4096} &= 1.6446899 & (|S - S_{4096}| &= 2.4414063 \times 10^{-4}) \\ S_{8192} &= 1.6448120 & (|S - S_{8192}| &= 1.2207031 \times 10^{-4}) \\ S_{16384} &= 1.6448730 & (|S - S_{16384}| &= 6.1035156 \times 10^{-5}) \\ S_{32768} &= 1.6449035 & (|S - S_{32768}| &= 3.0517578 \times 10^{-5}) \\ S_{65536} &= 1.6449188 & (|S - S_{65536}| &= 1.5258789 \times 10^{-5}) \end{aligned}$$

情報落ちを発生させない工夫が必要な時がある

演習

- 問題2-1

- 2次方程式の解を求めるプログラムを作成しなさい.
なお, 桁落ちへの対策を施すこと

- 問題2-2

- 11ページの無限級数の近似値を求めるプログラムを作成しなさい. なお, 情報落ちへの対策を施すこと

浮動小数点数

コンピュータ内部の実数：浮動小数点数表現が多い

符号 β 進 t 桁浮動小数点数

$$\bar{x} = \textcircled{\pm} \left(d_0 + \frac{d_1}{\beta} + \frac{d_2}{\beta^2} + \cdots + \frac{d_{t-1}}{\beta^{t-1}} \right) \times \beta^m$$

$$= \pm \boxed{d_0.d_1d_2\cdots d_{t-1}}_{\text{仮数}} \times \beta^{\text{指数 } m}$$

$(d_i \in \{0, 1, \dots, \beta - 1\}, L \leq \textcircled{m} \leq U, L, U : \text{定数})$

特によく使用：IEEE754形式

◆ IEEE754形式

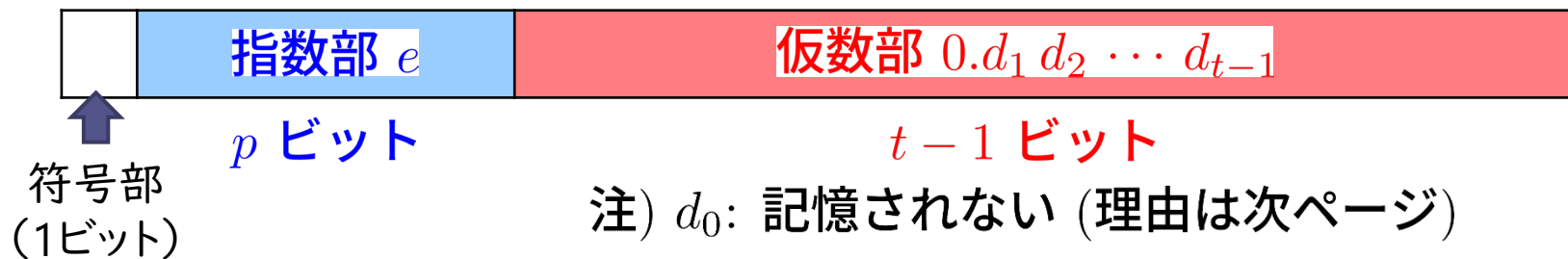
$$(-1)^s \times (d_0.d_1d_2 \cdots d_{t-1})_2 \times 2^m$$

$(s \in \{0, 1\}, m = e - (2^{p-1} - 1))$: バイアス表現, $0 \leq e \leq 2^p - 1$

バイアス値



メモリ上では...



- 単精度: $p = 8$ (バイアス値 = 127), $t = 24$ (仮数部 23 ビット)
- 倍精度: $p = 11$ (バイアス値 = 1023), $t = 53$ (仮数部 52 ビット)

補足) IEEE754-2008: 単精度は binary32, 倍精度は binary64 と呼ばれる。

◆ IEEE754形式における値表現

$$(-1)^s \times (d_0 \boxed{d_1 d_2 \cdots d_{t-1}})^f_{\text{とおく}} \times 2^{e-(2^{p-1}-1)}$$

- $e = 2^p - 1$ かつ $f \neq 0$ ならば, s の値によらず非数 (NaN) .
- $e = 2^p - 1$ かつ $f = 0$ ならば, $(-1)^s \infty$, すなわち正負の無限大.
- $0 < e < 2^p - 1$ ならば, $(-1)^s (1.f)_2 \times 2^{e-2^{p-1}+1}$ (正規化数).
- $e = 0$ かつ $f \neq 0$ ならば, $(-1)^s (0.f)_2 \times 2^{1-(2^{p-1}-1)}$ (非正規化数).
- $e = 0$ かつ $f = 0$ ならば, $(-1)^s 0$, すなわち正負の零. d_0 : 自動的に決定 (記憶の必要無)



- 絶対値最大数: $\pm(1.1 \cdots 1)_2 \times 2^{2^{p-1}-1} = \pm(2 - 2^{-t}) \times 2^{2^{p-1}-1}$
- 絶対値最小数 (正規化数): $\pm(1.0 \cdots 0)_2 \times 2^{1-(2^{p-1}-1)} = \pm 2^{2-2^{p-1}}$
- 絶対値最小数 (非正規化数): $\pm(0.0 \cdots 1)_2 \times 2^{1-(2^{p-1}-1)} = \pm 2^{3-2^{p-1}-t}$

◆ アンダーフロー・オーバーフロー

$$\pm \left(d_0 + \frac{d_1}{\beta} + \frac{d_2}{\beta^2} + \cdots + \frac{d_{t-1}}{\beta^{t-1}} \right) \times \beta^{e-(2^p-1)}$$

表現できる絶対値最大値 $x_{\max}$, 絶対値最小値 $x_{\min}$ が存在

(IEEE754 単精度: $x_{\max} = (2 - 2^{-23}) \times 2^{127} \approx 3.40 \times 10^{38}$, $x_{\min} = 2^{-149} \approx 1.40 \times 10^{-45}$)



ゆえに...

計算過程で絶対値が $x_{\max}$ を上回る, または $x_{\min}$ を下回ることもある

オーバーフロー (Overflow)

アンダーフロー (Underflow)

オーバーフロー・アンダーフローが起こると
プログラムはどう処理するのか?

```
public class Sample_ouflow {  
    public static void main(String[] args) {  
        float x, y;  
        x = 2.0F * Float.MAX_VALUE;  
        y = Float.MIN_VALUE / 2.0F;  
        System.out.println("2 * xmax = " + x);  
        System.out.println("xmin / 2 = " + y);  
    }  
}
```

絶対値最大値・最小値を
得ることが可能
(Short, Integer, Long, Double も有)



2 * xmax = Infinity
xmin / 2 = 0.0

オーバーフロー：無限大, アンダーフロー：0 と表現

言語によって取り扱いが違うので注意

◆ 浮動小数点の演算

加減算

- ① 指数を一致させるよう**仮数部を調整**
(指数の大きい方に小数点の位置を合わせる)
- ② **仮数部の加減算を実行**する
- ③ t桁浮動小数点数になるように**丸める**(後述)

乗除算

- ① 仮数の乗除算を行う
- ② **指数を更新**(乗算は加算, 除算は減算)する
- ③ t桁浮動小数点数になるように**丸める**(後述)

実数と同じく**交換律・結合律は成り立つのか?**

```
public class Sample_fpspro
{
    public static void main(String[] args) {
        double a = Math.pow(2,53);
        double b = 1.0D;
        double c = 1.0D;
        double d = a + b; d += c;
        double e = a; e += b + c;
        System.out.printf("(a + b) + c = %17.15e¥n",d);
        System.out.printf("a + (b + c) = %17.15e¥n",e);
    }
}
```



実行

```
(a + b) + c = 9.007199254740992e+15
a + (b + c) = 9.007199254740994e+15
```

計算順により
結果が異なる

結合律：成立しない

交換律は成立

◆ 丸め誤差

2進10桁浮動小数点の加算

$$\bar{x} = (1.100000000)_2 \times 2^{10}, \bar{y} = (1.100000000)_2 \times 2^1 \quad 11 \text{桁}$$


$$\begin{aligned} \bar{x} + \bar{y} &= (1.100000000)_2 \times 2^{10} + (0.0000000011)_2 \times 2^{10} \\ &= (1.1000000011)_2 \times 2^{10} \end{aligned}$$

11桁 ← 何らかの操作で10桁にする必要有

丸め

表現できる桁数より長い桁数の演算結果が得られたとき
その結果を表現できる桁数にするための操作.

丸めにより生じる誤差を丸め誤差と呼ぶ.

◆ IEEE754の丸めモード

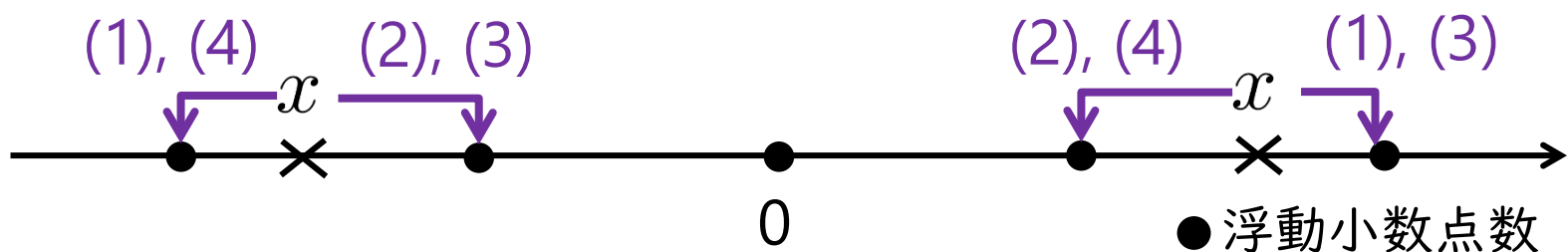
(1) 最近接丸め (偶数) : 標準

最も近くの表現できる値へ丸める. 表現可能な2つの値の
中間の値の場合, 一番低い桁が偶数(2進数の場合: 0)
となる方を採用

(2) 0 への丸め (towards zero)

(3) 正の無限大への丸め (towards positive infinity)

(4) 負の無限大への丸め (towards negative infinity)



◆ 計算機イプシロン

定義1.4 (計算機イプシロン)

$\mathbb{F}$: 浮動小数点数の集合とし, $x_f = \min\{x \in \mathbb{F} : x > 1\}$ とする.
このとき,

$$\varepsilon_M = x_f - 1.$$

を計算機イプシロン (マシンイプシロン) と呼ぶ.



最近接丸めモードの場合...

1. β 進 t 桁浮動小数点数: $\varepsilon_M = \beta^{1-t}$ (演習問題 (理論))

2. IEEE 浮動小数点数:

$$\varepsilon_M = 2^{-23} \approx 1.1920 \times 10^{-7} \text{ (単精度)}, = 2^{-52} \approx 2.220 \times 10^{-16} \text{ (倍精度)}$$

3. $1 \oplus \varepsilon_M > 1$ かつ $1 \oplus \frac{1}{\beta} \varepsilon_M = 1$ ($\oplus$: 浮動小数点演算の和)

定理1.1 (丸めの相対誤差評価)

- $\beta, t \in \mathbb{N}$ ($\beta \geq 2$). $\mathbb{F}_{\beta,t} = \{\beta\text{進 } t \text{ 桁浮動小数点数の集合}\}$
- $x_{\min}, x_{\max} \in \mathbb{F}_{\beta,t}$: 絶対値最小値, 最大値 (ただし $x_{\min} > 0$)
- $x_R \in \mathbb{F}_{\beta,t}$: $x \in \mathbb{R}$ ($x_{\min} < |x| < x_{\max}$) の最近接偶数への丸め

$\forall x \in \mathbb{R}$ ($x_{\min} < |x| < x_{\max}$) に対して,

$$\frac{|x - x_R|}{|x|} \leq \frac{1}{2} \varepsilon_M.$$

丸めを実行しただけで計算機イプシロンの割合だけ
誤差が混入する可能性がある

【証明概略】

$$\forall x \in \mathbb{R} (|x| > 0), \exists q \in \mathbb{Z}; \beta^{q-1} \leq |x| \leq \beta^q$$

アルキメデスの公理より導出： $\forall a \geq 0, \exists n \in \mathbb{N}; n - 1 \leq a < n$

- $|x| \geq 1$ のとき

$$\exists q \in \mathbb{N}; q - 1 \leq \frac{\log |x|}{\log \beta} \leq q \quad \Rightarrow \quad \beta^{q-1} \leq |x| < \beta^q$$

- $|x| < 1$ のとき

$$\exists q' \in \mathbb{N}; q' - 1 \leq -\frac{\log |x|}{\log \beta} \leq q' \quad \begin{matrix} q = 1 - q' \in \mathbb{Z} \\ \Rightarrow \end{matrix} \quad \beta^{q-1} < |x| \leq \beta^q$$

$$\exists q \in \mathbb{Z}; \beta^{q-1} \leq |x| \leq \beta^q \implies \beta^{q-1} \in \mathbb{F}_{\beta,t} \text{ and/or } \beta^q \in \mathbb{F}_{\beta,t}$$

区間 $[\beta^{q-1}, \beta^q]$: 浮動小数点数は間隔 β^{q-t} で分布

つまり...

$$x \text{ に最も近い浮動小数点数との距離} \leq \frac{1}{2}\beta^{q-t}$$

$$x_R: \text{最近偶数点への丸め} \quad \varepsilon_M = \beta^{1-t}$$

$$|x - x_R| \leq \frac{1}{2}\beta^{q-t} = \frac{1}{2}\beta^{q-1}\beta^{1-t} \leq \frac{1}{2}|x|\varepsilon_M$$

(証明終了)

最後に

$$98.7 \approx 1.1000101011001100110011001100110011001100110011001101 \times 2^6$$

$$78.9 \approx 1.0011101110011001100110011001100110011001100110011010 \times 2^6$$

(IEEE754倍精度浮動小数点数+丸めモード(1))



98.7+78.9 (の近似)

$$10.1100011001100110011001100110011001100110011001100111 \times 2^6$$



正規化+丸め

$$1.0110001100110011001100110011001100110011001100110100 \times 2^7$$



10進数に変換すると

177.600000000000002